# Niezawodność systemów informatycznych

Raport numer 2 (P2)

Testy strukturalne: pokrycie kodu

Trudov Mykhailo, Mykola Blavitskyi, Oleh Stasiuk
Zespół G

21 października 2026

## SUT

System Under Test pozostaje bez zmian względem raportu P1: system zarządzania kontem bankowym (wariant zespołu G, plik `zespol_G.py`). Udostępnia utworzenie konta, wpłatę, wypłatę, przelew wewnętrzny, naliczanie odsetek, historię, wyciąg miesięczny oraz przelew zewnętrzny przez symulowany procesor płatności.

Kod produkcyjny **nie był poprawiany** — testy strukturalne mierzą i dokumentują faktyczne zachowanie, w tym usterki zidentyfikowane w P1 (podwójny wpis historii w `wplac`, wpłata na konto zablokowane, przelew na zablokowane konto odbiorcy, brak sprawdzenia typu kwoty w `wyplac`).

## Repozytorium

Link do repozytorium: https://github.com/SadAndAlone/SUT_zespolG_MT_OS_MB

Testy jednostkowe P2 znajdują się w katalogu `tests/` (`test_structural_wplac.py`, `test_structural_wyplac.p`
`test_structural_przelew.py`).

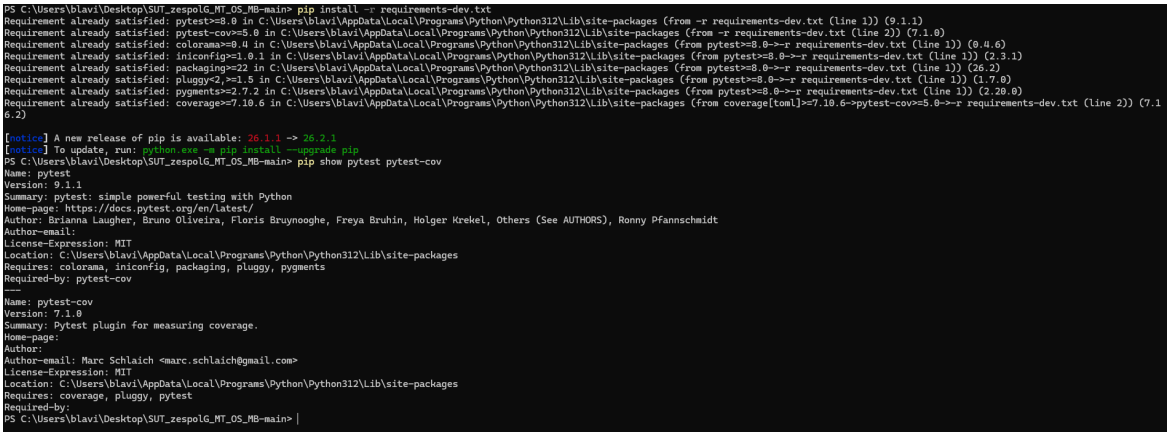
## Cel zajęć nr 2

Na zajęciach nr 1 SUT testowano eksploracyjnie, bez planu opartego o strukturę kodu. Na zajęciach nr 2 zastosowano strategię przeciwną: testy białoskrzynkowe planowane na podstawie instrukcji warunkowych, tak aby świadomie wykonać każdą gałąź wybranych metod (sukces, walidacja kwoty, brak środków, limit, konto zablokowane, wyjątki).

Pokrycie kodu traktujemy jako warunek konieczny, ale niewystarczający dobrych testów: każdy przypadek zawiera asercję (`assert` albo `pytest.raises`), a nie tylko wywołanie metody.

## 1 Narzędzia

Zainstalowano `pytest` oraz wtyczkę `pytest-cov` (pomiar pokrycia na bazie `coverage.py`), zgodnie z kartą zajęć.




**Rysunek 1:** Instalacja narzędzi i potwierdzenie wersji: `pip install -r requirements-dev.txt` oraz `pip show pytest pytest-cov` (pytest 9.1.1, pytest-cov 7.1.0, Python 3.12.10).

Uruchamianie testów z pomiarem pokrycia instrukcji i gałęzi:

```
pytest -m "not pelny" --cov=zespol_G --cov-report=term-missing --cov-branch
pytest --cov=zespol_G --cov-report=term-missing --cov-branch --cov-report=html
```

Flaga `term-missing` wypisuje numery niepokrytych linii w terminalu, `-cov-branch` włącza pokrycie gałęzi, a `html` zapisuje raport w katalogu `htmlcov/`.

## 2 Wybrane metody i przypadki testowe

Wybrano trzy metody codziennego ruchu banku (zgodnie z profilem operacyjnym z P1 oraz przykładem z karty P2): `wplac`, `wyplac`, `przelew`.

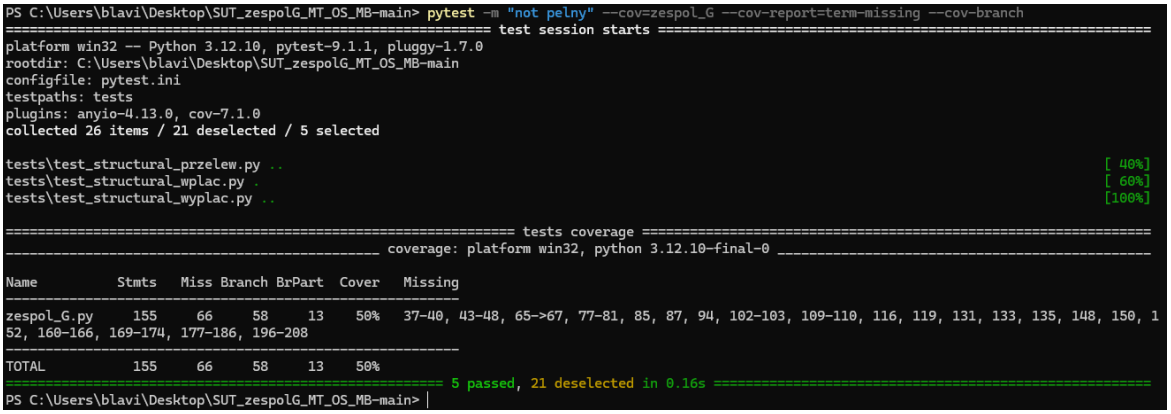
**Tabela 1:** Zestaw testów strukturalnych (parametrize rozwija osobne przypadki kwoty).

| Metoda | Plik testowy | Funkcje | Przypadki pytest |
|---|---|---|---|
| `wplac` | `tests/test_structural_wplac.py` | 5 | 7 |
| `wyplac` | `tests/test_structural_wyplac.py` | 9 | 10 |
| `przelew` | `tests/test_structural_przelew.py` | 8 | 9 |
| Razem | | 22 | 26 |

Przypadki obejmują ścieżkę sukcesu oraz gałęzie wyjątkowe wymagane na karcie: konto nieistniejące, kwota niedodatnia, niewystarczające środki, przekroczony limit, konto zablokowane. Dodatkowo udokumentowano usterki P1 asercjami (m.in. `len(historia) == 2` po wpłacie; wpłata na zablokowane konto nie zgłasza wyjątku; przelew na zablokowanego odbiorcę przechodzi).

## 3 Pomiar pokrycia — faza 1 (przed uzupełnieniem)

Faza 1 to zestaw bazowy (ścieżki sukcesu oraz blokada nadawcy): `pytest -m "not pelny"`. Dwadzieścia jeden przypadków walidacji jest wówczas celowo wyłączonych znacznikiem `pelny`.




**Rysunek 2:** Faza 1 — pokrycie *przed* dopisaniem testów zamykających gałęzie. Polecenie: `pytest -m "not pelny-cov=zespol_G -cov-report=term-missing -cov-branch`. Wynik: 5 passed, 21 deselected, Cover 50%. Kolumna `Missing` zawiera m.in. linie 116, 119, 131, 133, 135, 148, 150, 152 (walidacja typu/kwoty, środki, limity).

**Tabela 2:** Pokrycie w fazie 1 (cały plik `zespol_G.py`).

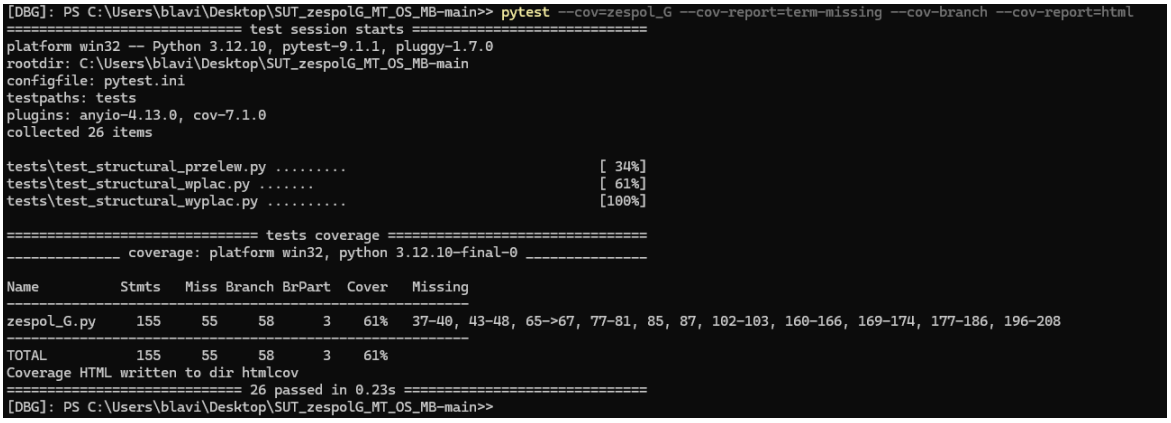
| Zakres | Instrukcje | Gałęzie | Raport łączony |
|---|---|---|---|
| `zespol_G.py` | $89/155 = 57\%$ | $17/58 = 29\%$ | 50% |
| `wplac` | 80% | 50% | — |
| `wyplac` | 77% | 62% | — |
| `przelew` | 82% | 62% | — |

Różnica między pokryciem instrukcji a gałęzi jest tu dobrze widoczna: ścieżka sukcesu w `wplac` wykonuje większość linii ciała metody (80% stmt), ale nie wchodzi w `TypeError` ani `kwota <= 0`, więc gałęzie zostają na 50%. To dokładnie sytuacja opisywana na karcie zajęć.

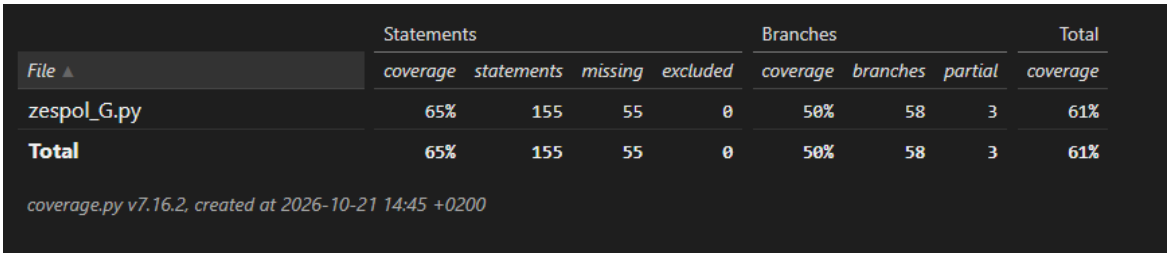
## 4 Pomiar pokrycia — faza 2 (po uzupełnieniu)

Na podstawie kolumny `Missing` z fazy 1 dopisano testy oznaczone `@pytest.mark.pelny`: nieistniejące konto, zły typ kwoty, kwota niedodatnia, niewystarczające środki, limity, `wyplac(None)` oraz przelew na zablokowane konto odbiorcy.

Przy teście limitu dziennego okazało się, że warunek w SUT to `suma + kwota > 5000`, a nie `>=`. Wypłata równa `LIMIT_DZIENNY_WYPLAT` przechodzi — test z kwotą dokładnie 5000 nie zgłaszał `ValueError`. Poprawiono przypadek na $5000 + 1$ oraz dodano scenariusz dwóch wypłat, które razem przekraczają limit.




**Rysunek 3:** Faza 2 — pełny zestaw po uzupełnieniu braków. Polecenie: `pytest -cov=zespol_G -cov-report=term-missing -cov-branch -cov-report=html`. Wynik: 26 passed, Cover 61%. Z `Missing` zniknęły linie ciał `wplac`/`wyplac`/`przelew`; pozostają metody poza zakresem P2 (m.in. 160–166, 177–186, 196–208).



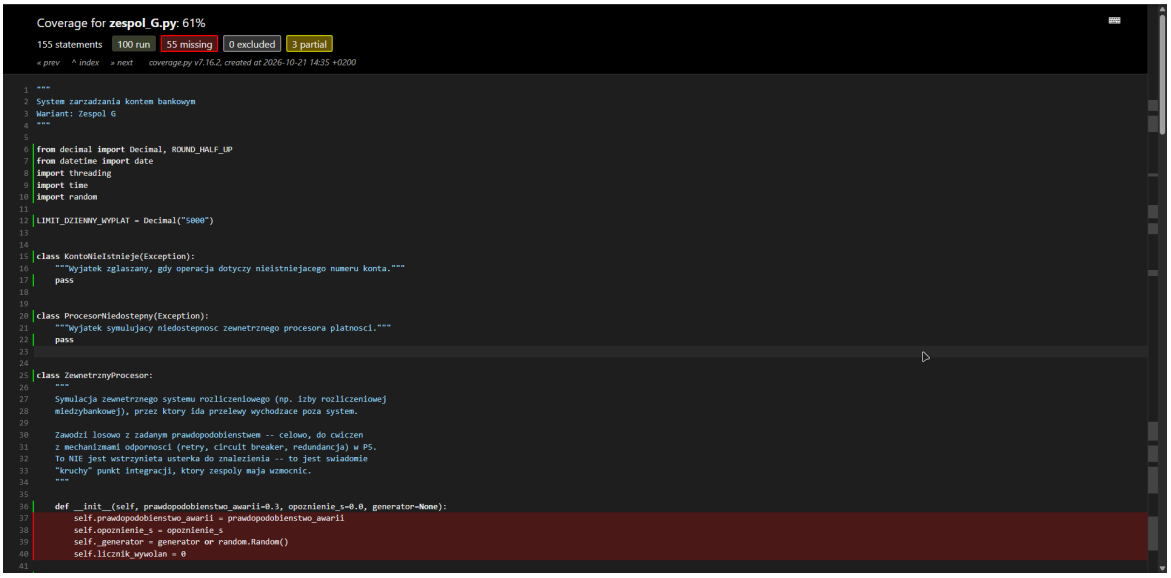

**Rysunek 4:** Raport HTML (`htmlcov/index.html`) po fazie 2: pokrycie instrukcji 65% (100/155), pokrycie gałęzi 50% (29/58), wynik łączony 61%. Osobne kolumny Statements i Branches spełniają wymaganie karty: zapisać wynik procentowy dla obu poziomów.

**Tabela 3:** Pokrycie w fazie 2 — porównanie z fazą 1.

| Zakres | Stmt (f1) | Branch (f1) | Stmt (f2) | Branch (f2) |
|---|---|---|---|---|
| `zespol_G.py` | 57% | 29% | 65% | 50% |
| `wplac` | 80% | 50% | **100%** | **100%** |
| `wyplac` | 77% | 62% | **100%** | **100%** |
| `przelew` | 82% | 62% | **100%** | **100%** |

## 5 Niepokryte fragmenty (świadomie pominięte)

Raport HTML z podświetleniem źródła pozwala wskazać konkretne linie, które nie zostały wykonane. Czerwone bloki nie są przeoczeniem testów trzech wybranych metod — leżą poza zakresem P2.




**Rysunek 5:** Widok `htmlcov/zespol_G_py.html`: 100 linii wykonanych, 55 brakujących, 3 gałęzie częściowe. Czerwony blok `ZewnetrznyProcesor.__init__` (linie 37–40) należy do kruchego procesora płatności przewidzianego na blok P5. Zielone importy i definicje wyjątków zostały załadowane przy starcie testów.

Świadomie pominięto:

- `przelew_zewnetrzny` oraz `ZewnetrznyProcesor` (linie 37–48, 196–208) — losowa awaria procesora, zadanie na blok P5; w P1 ten obszar wyłączono z baseline.
- `nalicz_odsetki` (160–166), `historia_w_okresie` (169–174), `wyciag_miesieczny` (177–186) — nie należą do trzech wybranych metod.
- `_numer_poprawny` / gałąź błędu w `utworz_konto` (77–81, 85, 87) — usterka P1 (suma kontrolna nieużywana); fixture tworzy konta poprawnym formatem 10-cyfrowym.
- `odblokuj_konto` (102–103) oraz gałąź `dodaj_wpis` z jawną datą (65 → 67) — poza wybranymi metodami; testy używają `date.today()`.

Dlatego pokrycie całego pliku na poziomie 61% jest oczekiwane i nie oznacza słabego zestawu dla `wplac`/`wyplac`/`przelew` (tam 100% instrukcji i 100% gałęzi).

## 6 Wnioski

- Testy strukturalne z asercjami **potwierdziły** usterki z P1: podwójna historia wpłaty, brak blokady przy `wplac`, przelew na zablokowane konto odbiorcy, `wyplac(None)` kończy się `InvalidOperation` zamiast `TypeError`.
- Przy domykaniu gałęzi limitu dziennego znaleziono zachowanie niezgodne z naiwnym oczekiwaniem „równość limitu = błąd”: operator w kodzie to `>`, więc wypłata równa 5000 jest dozwolona. Tego nie odnotowano w P1.
- Porównanie faz pokazuje, po co mierzy się gałęzie, a nie tylko instrukcje: faza 1 dawała już wysokie stmt na wybranych metodach przy wyraźnie niższym branch. Dopiero testy wyjątków domknęły obie strony każdego `if`.
- 100% pokrycia gałęzi w trzech metodach **nie** oznacza braku usterek — testy celowo dokumentują błędne zachowanie tam, gdzie specyfikacja banku wymagałaby wyjątku. Pokrycie mówi, który kod się wykonał, a nie czy wynik jest zgodny ze specyfikacją; o tym decyduje asercja.

Środowisko (`pytest`, `coverage`) oraz napisane testy pozostają w repozytorium jako punkt wyjścia do zajęć nr 3 (testy mutacyjne).

## 7 Pytania kontrolne

1. **Czym różni się pokrycie instrukcji od pokrycia gałęzi? Podaj przykład kodu, dla którego oba poziomy dają różny wynik.**

   Pokrycie instrukcji to odsetek linii (statementów) wykonanych przynajmniej raz. Pokrycie gałęzi to odsetek wychodzeń z decyzji (prawda *i* fałsz każdego `if`). Dla kodu:

   ```
   if x:
       A()
   ```

   jeden test z `x = True` daje 100% instrukcji (wykonano `if` i `A`), ale tylko 50% gałęzi — nie sprawdzono strony fałszywej. W naszym SUT: faza 1 na `wplac` to 80% stmt i 50% branch (Rys. 2).

2. **Dlaczego 100% pokrycia instrukcji nie gwarantuje wykrycia wszystkich usterek?**

   Pokrycie nie sprawdza, czy wynik został porównany z oczekiwaniem. Test bez `assert` może wywołać całą metodę, uzyskać 100% stmt i nie zauważyć, że `wplac` zapisuje historię dwukrotnie. Ponadto usterka może leżeć w wartości brzegowej (u nas: limit `>` zamiast `>=`) albo w nieuruchomionej gałęzi, której instrukcja „sukcesu” już została policzona. Stąd na karcie: pokrycie jest miarą kompletności, nie skuteczności.

3. **Co to jest MC/DC i w jakim kontekście stosuje się ten standard?**

   MC/DC (*Modified Condition/Decision Coverage*) wymaga, aby każdy warunek atomowy w złożonej decyzji (np. `a and b`) przyjął wartość prawda i fałsz *oraz* niezależnie wpływał na wynik całej dezycji. To rygorystyczniejszy poziom niż zwykłe pokrycie warunków. Stosuje się go m.in. w oprogramowaniu lotniczym (norma DO-178C).

4. **Jaka jest różnica między testem, który podnosi pokrycie, a testem, który weryfikuje poprawność?**

   Test „na pokrycie” jedynie uruchamia fragment kodu — licznik coverage rośnie niezależnie od tego, czy saldo, historia i wyjątek są poprawne. Test weryfikujący ma asercję względem specyfikacji (albo względem świadomie udokumentowanego błędu): `assert saldo == Decimal("1100")`, `assert len(konto.historia) == 2`, `pytest.raises(KontoZablokowane, ...)`. Drugi rodzaj wykrywa usterki; pierwszy — tylko zwiększa procent.